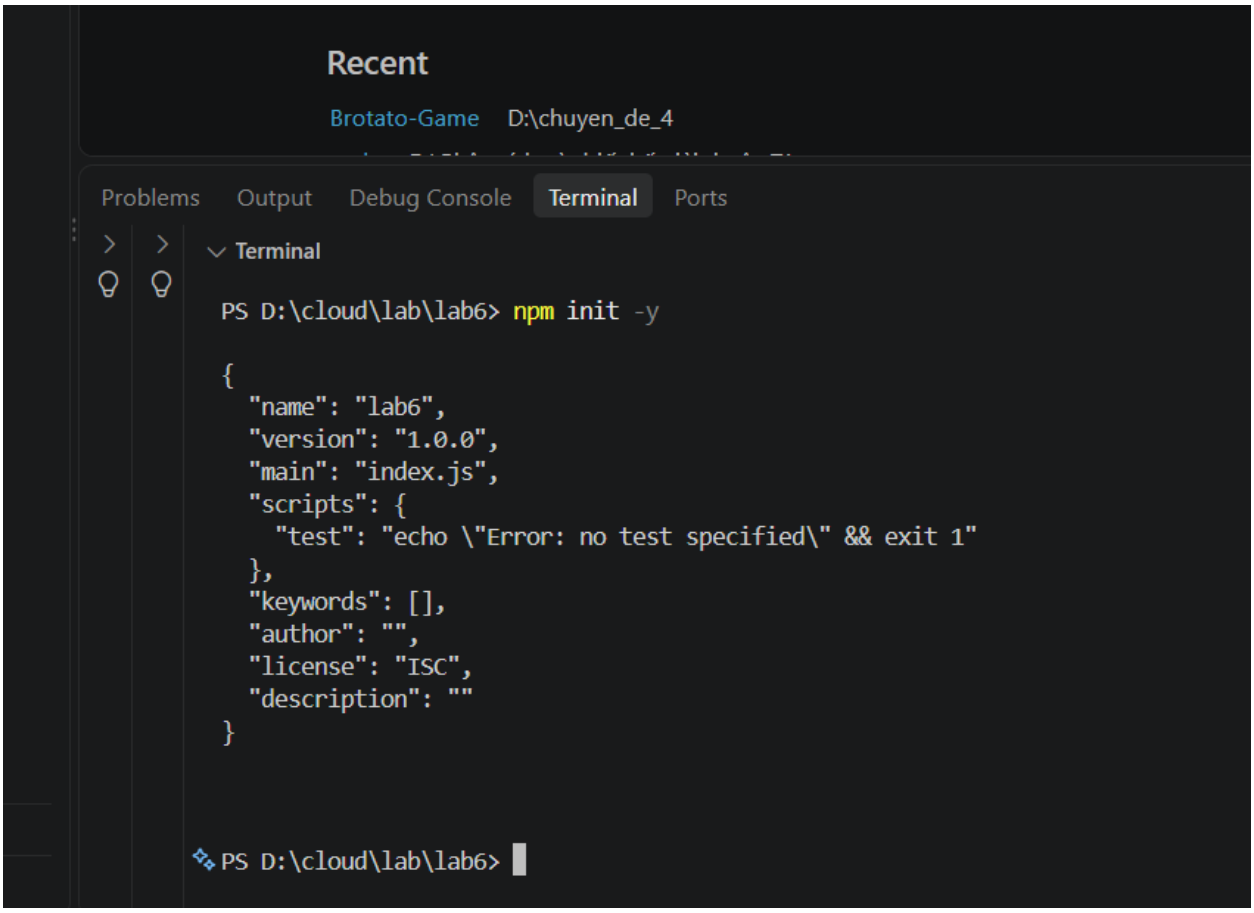
LAB 6 : HOW TO PERFORM CRUD OPERATIONS WITH MONGOOSE AND MONGODB ATLAS

Họ Và Tên : Trần Quang Tuấn

Mã Sinh Viên : 23IT300

Step 1 . Setting Up the Project

1. Khởi tạo dự án
   Lệnh thực thi trong Terminal: npm init -y
   Giải thích: Lệnh này tự động tạo tệp package.json ở thư mục gốc để quản lý thông tin dự án (tên dự án :ab6, phiên bản 1.0.0, kiểu module commonjs).



2. **Cài đặt thư viện**
   Lệnh thực thi trong Terminal: npm install express mongoose

Giải thích:

- **express**: Web framework phục vụ việc dựng Web Server và định tuyến các đường dẫn API.
- **mongoose**: Thư viện ODM (Object Data Modeling) giúp kết nối và thao tác với cơ sở dữ liệu MongoDB thông qua các Schema và Model

```
PS D:\cloud\lab\lab6> npm install express mongoose

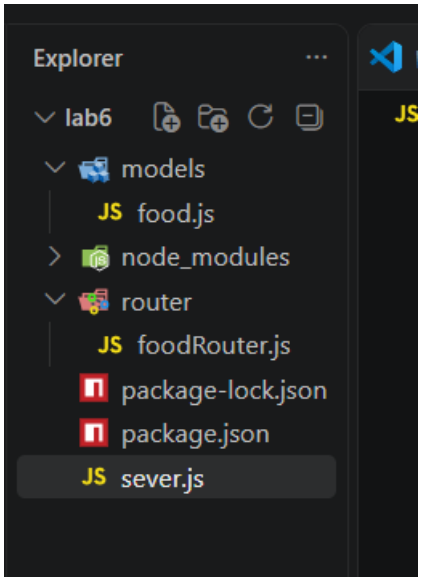
added 83 packages, and audited 84 packages in 24s

29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\cloud\lab\lab6>
```

3. Cấu trúc thư mục dự án

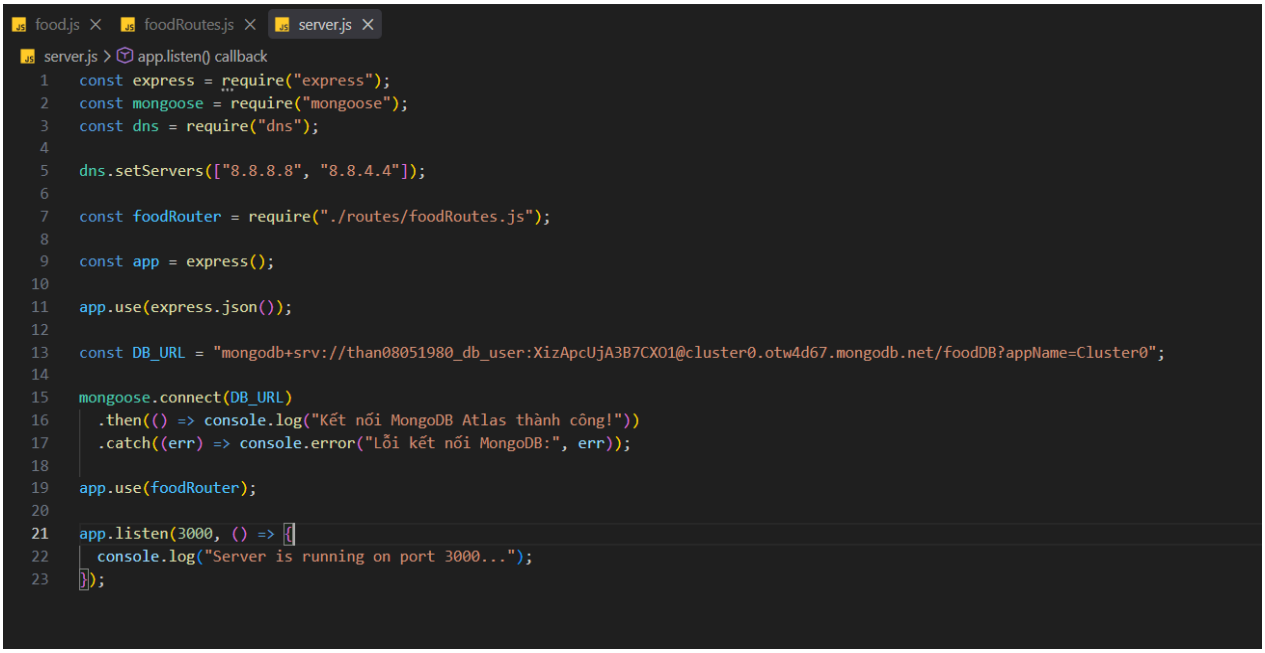
- **models/**: Thư mục chứa tệp food.js dùng để định nghĩa cấu trúc dữ liệu (Schema).
- **routes/**: Thư mục chứa tệp foodRoutes.js quản lý các đường dẫn API (GET, POST, PATCH, DELETE).
- **node_modules/**: Thư mục chứa mã nguồn của các thư viện đã cài đặt.
- **package.json & package-lock.json**: Tệp quản lý thông tin dự án và danh sách thư viện phụ thuộc.
- **server.js**: Tệp điểm đầu (entry point) khởi chạy server và kết nối CSDL.



Step 2 . Setting Up the Server

Khởi tạo Web Server bằng Express và thiết lập kết nối đến MongoDB Atlas.

Mã nguồn server.js:



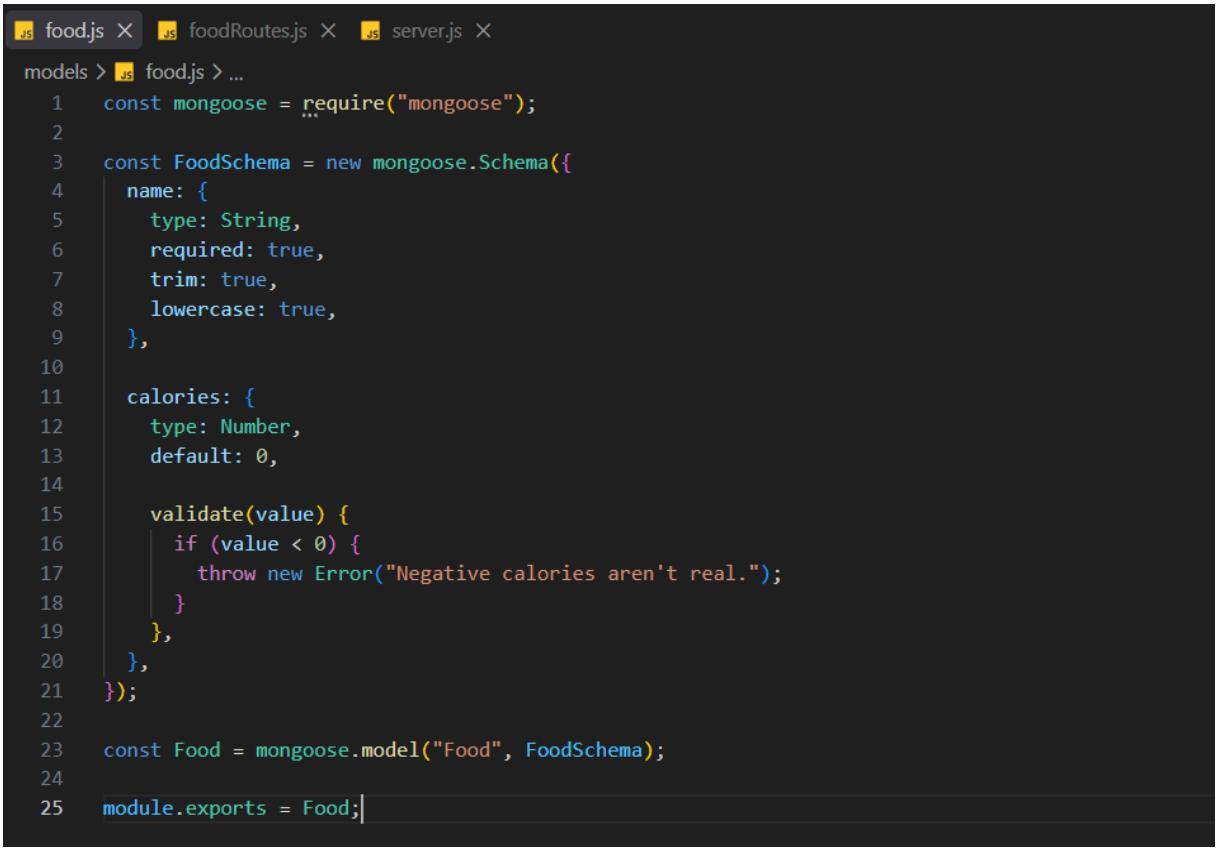
Giải thích chi tiết mã nguồn:

- **const express = require("express");**: Nạp thư viện Express vào dự án.
- **const mongoose = require("mongoose");**: Nạp thư viện Mongoose.
- **const dns = require("dns");**: Nạp module DNS mặc định của Node.js.
- **dns.setServers(["8.8.8.8", "8.8.4.4"]);**: Chỉ định Node.js sử dụng DNS Google nhằm khắc phục lỗi phân giải tên miền SRV (querySrv ECONNREFUSED) khi kết nối MongoDB Atlas.
- **const foodRouter = require("./routes/foodRoutes.js");**: Nhập tệp định tuyến chứa các route xử lý món ăn.
- **const app = express();**: Tạo một thể hiện ứng dụng Express.
- **app.use(express.json());**: Middleware giúp Express tự động đọc và xử lý dữ liệu dạng JSON được gửi từ client trong request.body.
- **const DB_URL = "...";**: Chuỗi kết nối MongoDB Atlas bao gồm username, password, địa chỉ Cluster và chỉ định database foodDB.
- **mongoose.connect(DB_URL)...**: Khởi tạo kết nối bất đồng bộ tới CSDL, in thông báo thành công hoặc bắt lỗi nếu kết nối thất bại.
- **app.use(foodRouter);**: Đăng ký router món ăn vào ứng dụng Express.
- **app.listen(3000, ...);**: Lắng nghe yêu cầu kết nối ở cổng 3000.

Step 3 . Building the Schema

Định nghĩa cấu trúc dữ liệu và các quy tắc kiểm tra (validation) cho tập hợp foods.

Mã nguồn models/food.js:



Giải thích chi tiết mã nguồn:

- **const mongoose = require("mongoose");**: Nạp thư viện Mongoose.
- **const FoodSchema = new mongoose.Schema({...});**: Tạo một Schema mới để quy định cấu trúc dữ liệu.
    - **Trường name**:
        - type: String: Quy định dữ liệu phải là dạng chuỗi.
        - required: true: Trường bắt buộc, không được để trống.
        - trim: true: Tự động cắt bỏ các khoảng trắng thừa ở đầu và cuối chuỗi.
        - lowercase: true: Chuyển tất cả ký tự thành chữ thường trước khi lưu vào CSDL.
    - **Trường calories**:
        - type: Number: Dữ liệu dạng số.
        - default: 0: Giá trị mặc định là 0 nếu người dùng không nhập.
        - validate(value): Hàm kiểm tra tùy chỉnh, nếu giá trị < 0 sẽ báo lỗi "Negative calories aren't real.".

- **const Food = mongoose.model("Food", FoodSchema);**: Tạo Model tên Food, Mongoose sẽ tự động liên kết tới collection tên foods trong MongoDB.
- **module.exports = Food;**: Xuất Model để sử dụng ở các tệp khác.

Step 4 . Building the Read Route

Định nghĩa API lấy danh sách tất cả các món ăn từ CSDL.

Mã nguồn trong routes/foodRoutes.js (Thao tác Read):



Giải thích chi tiết mã nguồn:

- **const express = require("express");**: Nạp thư viện Express.
- **const foodModel = require("../models/food");**: Import Model Food để truy vấn CSDL.
- **const app = express();**: Tạo đối tượng Router.
- **app.get("/foods", async (request, response) => {...});**: Lắng nghe phương thức HTTP GET tại đường dẫn /foods.
- **const foods = await foodModel.find({});**: Sử dụng phương thức find({}) với đối tượng rỗng để truy vấn toàn bộ bản ghi có trong tập hợp foods.
- **response.status(200).send(foods);**: Trả về danh sách món ăn thu được dưới dạng JSON cùng mã trạng thái 200 OK.
- **catch (error) { response.status(500).send(error); }**: Bắt và trả về lỗi hệ thống với mã 500 Internal Server Error nếu việc truy vấn thất bại.
- **module.exports = app;**: Xuất Router để tích hợp vào server.js.

Step 5 . Building the Create Route

Định nghĩa API thêm mới một món ăn vào CSDL.

Mã nguồn trong routes/foodRoutes.js (Create):

```
app.post("/food", async (request, response) => {
  const food = new foodModel(request.body);

  try {
    await food.save();
    response.status(201).send(food);
  } catch (error) {
    response.status(500).send(error);
  }
});

module.exports = app;
```

Giải thích chi tiết mã nguồn:

- **app.post("/food", async (request, response) => {...});**: Lắng nghe phương thức HTTP POST tại endpoint /food.
- **const food = new foodModel(request.body);**: Khởi tạo một đối tượng món ăn mới từ dữ liệu người dùng gửi lên trong request.body.
- **await food.save();**: Gọi phương thức save() bất đồng bộ để lưu tài liệu mới vào MongoDB Atlas.
- **response.status(201).send(food);**: Trả về dữ liệu vừa tạo thành công (bao gồm mã _id do MongoDB cấp) kèm mã 201 Created.
- **catch (error) { response.status(500).send(error); }**: Xử lý lỗi (như vi phạm ràng buộc Schema) và phản hồi mã 500.

Step 6 . Building the Update Route

Định nghĩa API cập nhật thông tin của món ăn dựa theo ID.

Mã nguồn trong routes/foodRoutes.js (Update):

```
app.patch("/food/:id", async (request, response) => {
  try {
    const food = await foodModel.findByIdAndUpdate(
      request.params.id,
      request.body,
      { new: true, runValidators: true }
    );
    if (!food) {
      return response.status(404).send("Không tìm thấy món ăn");
    }
    response.send(food);
  } catch (error) {
    response.status(500).send(error);
  }
});

module.exports = app;
```

Giải thích chi tiết mã nguồn:

- **app.patch("/food/:id", async (request, response) => {...});**: Lắng nghe phương thức HTTP PATCH tại endpoint /food/:id với :id là tham số chứa ID của món ăn cần sửa.
- **await foodModel.findByIdAndUpdate(...)**: Hàm tìm kiếm và cập nhật dữ liệu theo ID với 3 tham số:
    1. request.params.id: Lấy ID truyền từ thanh URL.
    2. request.body: Dữ liệu mới cần cập nhật.
    3. { new: true, runValidators: true }: Tùy chọn yêu cầu Mongoose trả về đối tượng sau khi đã cập nhật (new: true) và ép buộc chạy lại các quy tắc kiểm tra ràng buộc Schema (runValidators: true).
- **if (!food) { return response.status(404).send("Không tìm thấy món ăn"); }**: Kiểm tra nếu không tồn tại món ăn với ID trên thì phản hồi mã 404 Not Found.
- **response.send(food);**: Phản hồi dữ liệu món ăn đã được chỉnh sửa cho client.

Step 7 . Building the Delete Route

Định nghĩa API xóa món ăn khỏi CSDL dựa theo ID.

Mã nguồn trong routes/foodRoutes.js (Delete):

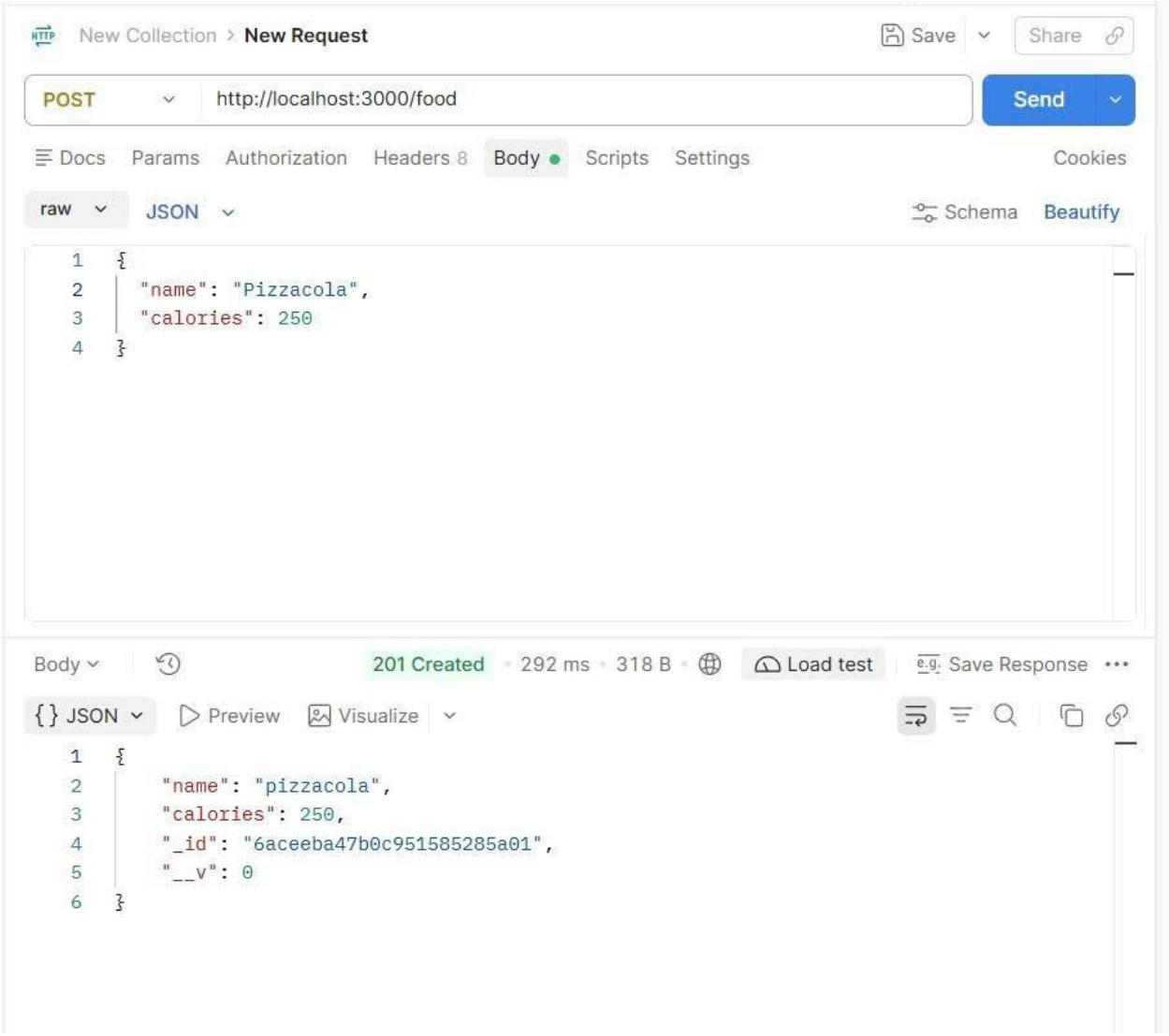
```
42
43  app.delete("/food/:id", async (request, response) => {
44    try {
45      const food = await foodModel.findByIdAndDelete(request.params.id);
46
47      if (!food) {
48        return response.status(404).send("Không tìm thấy món ăn để xóa");
49      }
50      response.status(200).send({ message: "Xóa thành công!" });
51    } catch (error) {
52      response.status(500).send(error);
53    }
54  });
55
56  module.exports = app;
57
```

Giải thích chi tiết mã nguồn:

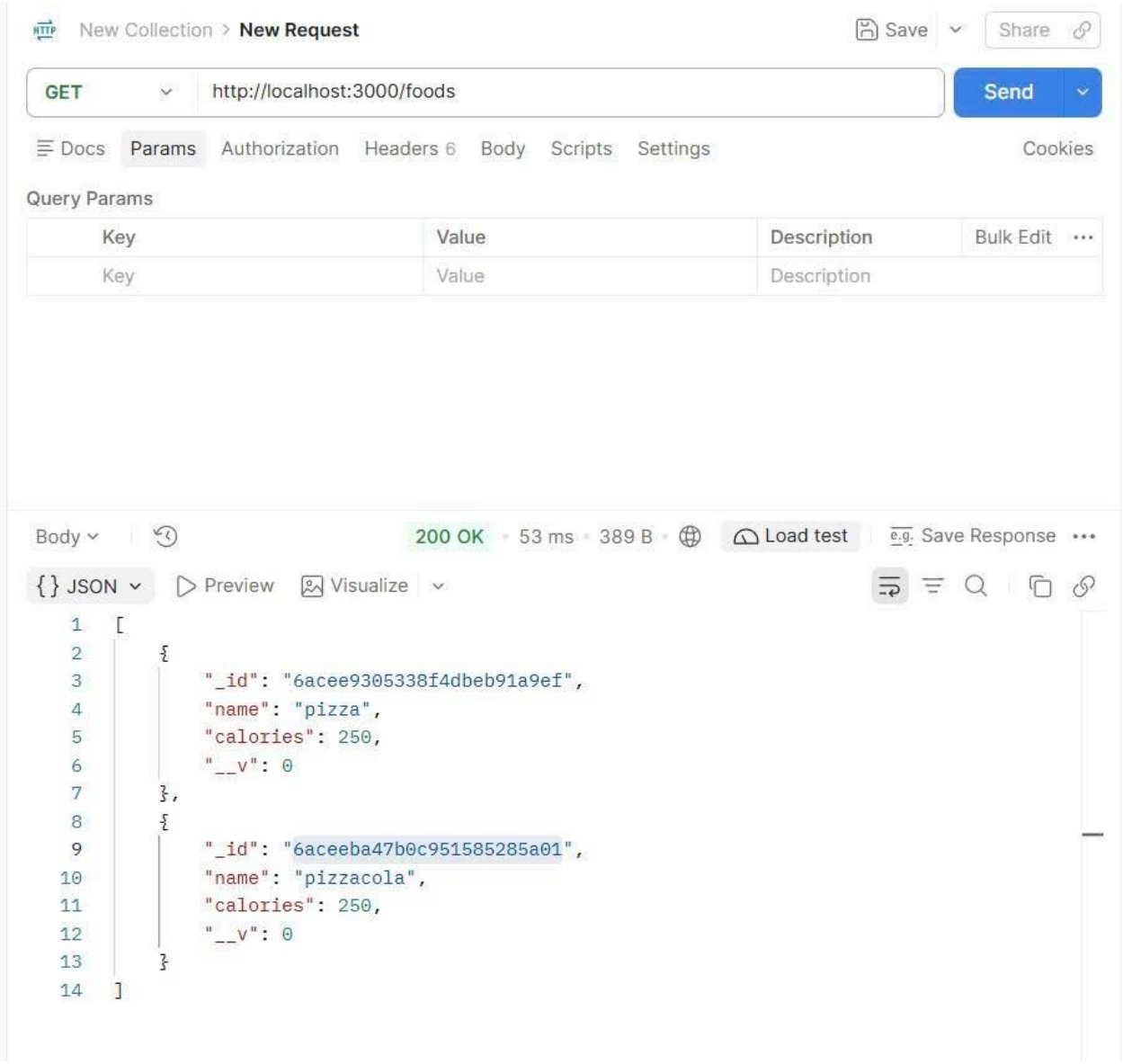
- **app.delete("/food/:id", async (request, response) => {...});**: Lắng nghe phương thức HTTP DELETE tại endpoint /food/:id.
- **await foodModel.findByIdAndDelete(request.params.id);**: Tìm tài liệu có _id khớp với request.params.id và xóa khỏi CSDL.
- **if (!food) { return response.status(404).send("Không tìm thấy món ăn để xóa"); }**: Nếu không tìm thấy món ăn cần xóa thì trả về lỗi 404 Not Found.
- **response.status(200).send({ message: "Xóa thành công!" });**: Khi xóa thành công, trả về mã 200 OK cùng thông báo xác nhận dạng JSON.

Postman CRUD

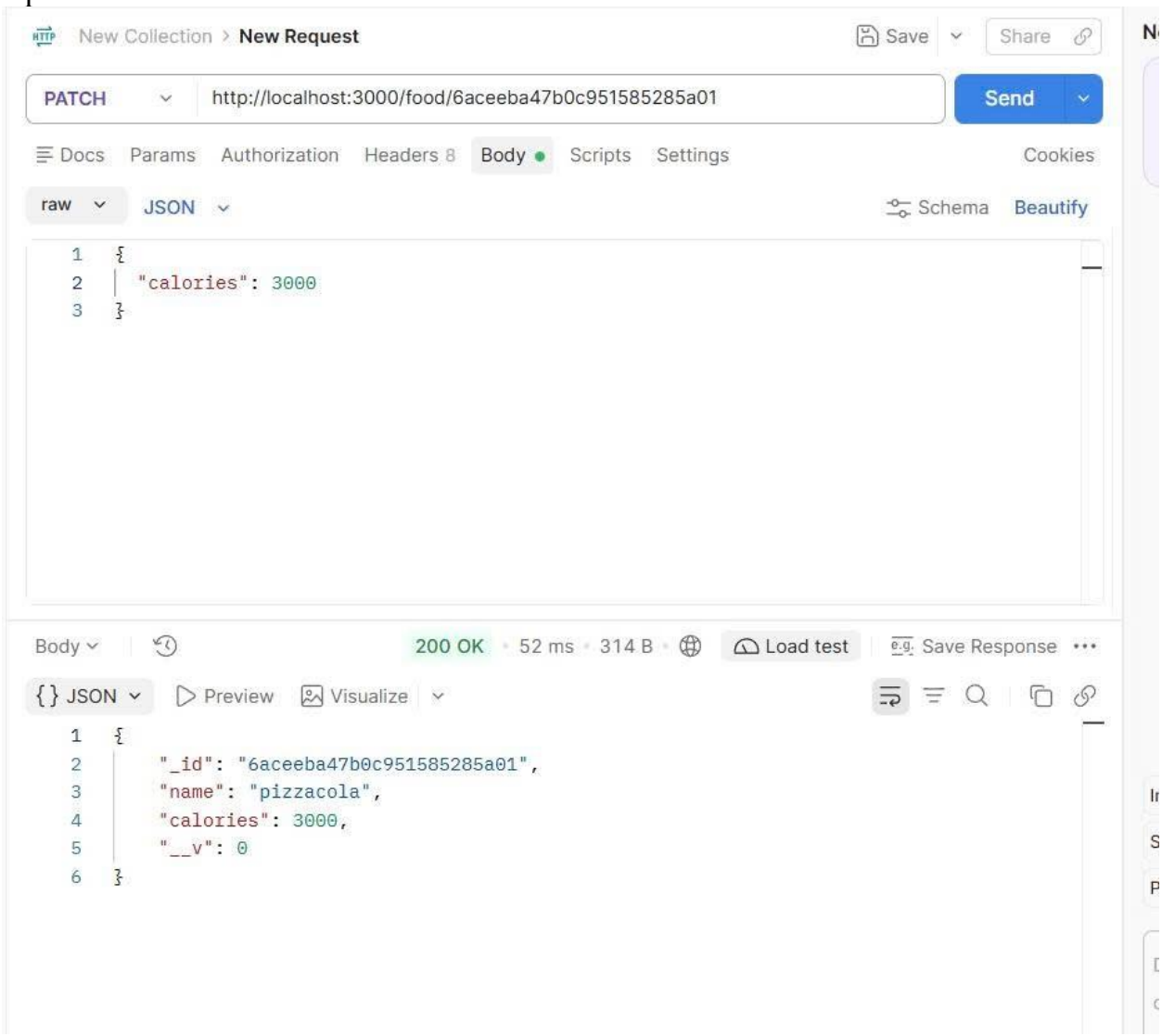
Create


New Collection > New Request
Save
Share
POST
http://localhost:3000/food
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
1 {
2 "name": "Pizzacola",
3 "calories": 250
4 }
Body
201 Created
292 ms
318 B
Load test
Save Response
{ } JSON
Preview
Visualize
1 {
2 "name": "pizzacola",
3 "calories": 250,
4 "_id": "6aceeba47b0c951585285a01",
5 "__v": 0
6 }

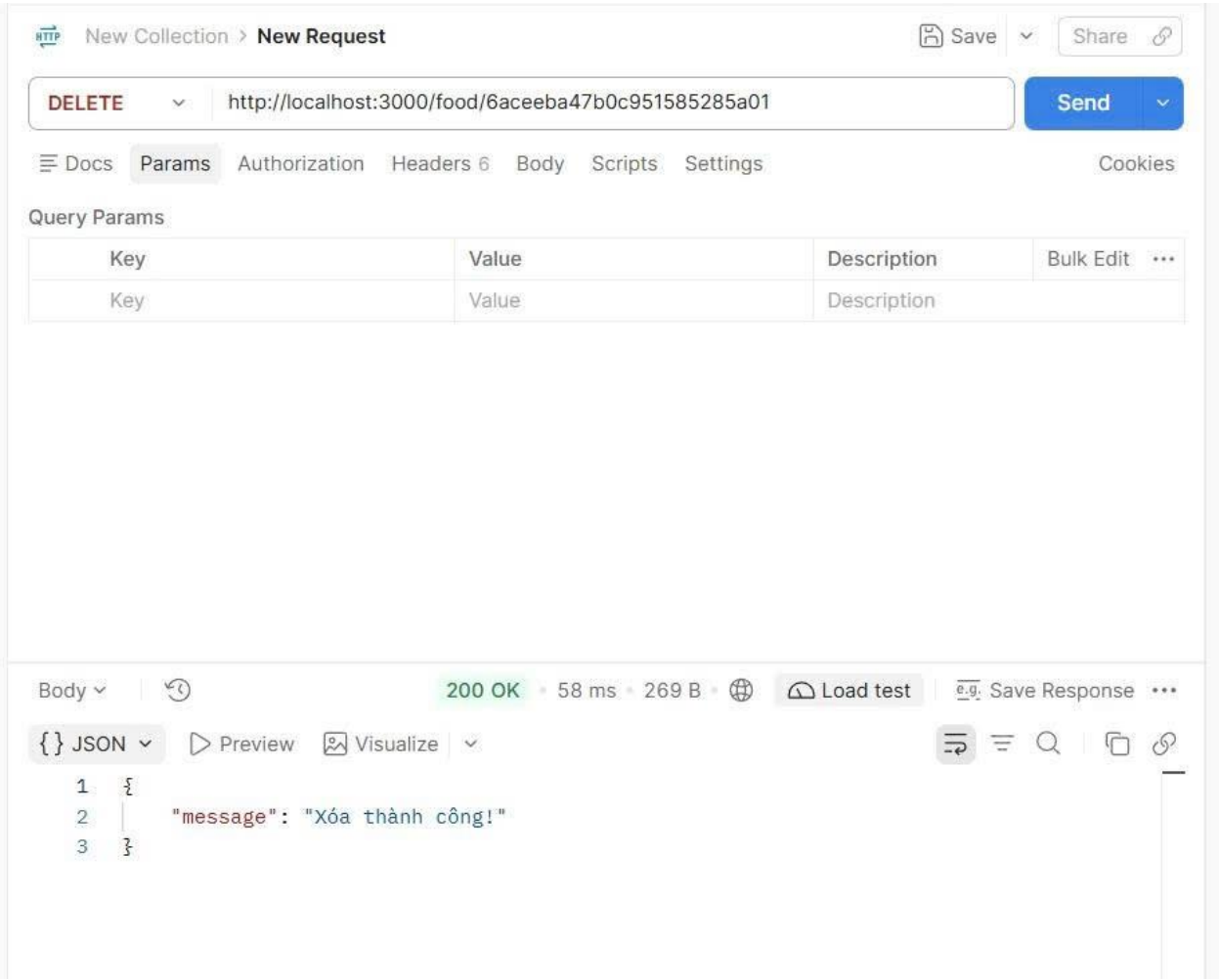
Get


New Collection > New Request
Save
Share
GET
http://localhost:3000/foods
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
200 OK
53 ms
389 B
Load test
Save Response
{ } JSON
Preview
Visualize
[
{
"_id": "6acee9305338f4dbeb91a9ef",
"name": "pizza",
"calories": 250,
"__v": 0
},
{
"_id": "6aceeba47b0c951585285a01",
"name": "pizzacola",
"calories": 250,
"__v": 0
}
]

Update


New Collection > New Request
Save
Share
PATCH
http://localhost:3000/food/6aceeba47b0c951585285a01
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"calories": 3000
}
Body
200 OK
52 ms
314 B
Load test
Save Response
{ } JSON
Preview
Visualize
{
"_id": "6aceeba47b0c951585285a01",
"name": "pizzacola",
"calories": 3000,
"__v": 0
}

Delete


New Collection > New Request
Save
Share
DELETE
http://localhost:3000/food/6aceeba47b0c951585285a01
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
200 OK
58 ms
269 B
Load test
Save Response
{} JSON
Preview
Visualize
1 {
2 "message": "Xóa thành công!"
3 }